



CY TECH
IPS Spring Program 2026
Rain Prediction System

Done by: Youssef ALY – Isela Allhassa – Pietro Vinsten

Supervised by: Prof. Souhila Arib

Acknowledgement

We would like to thank Professor Souhila Arib for her guidance and support through this semester's project, she offered us all the support and encouragement to finish our project, she gave us all her time for focusing on our needs for the project, helped us with many facilities to facilitate our progress, also we would like to thank all our professors during this academic year for their support and efforts for our class and for his support. I also appreciate my colleagues for their collaboration and support, It was a great pleasure working on this project as our IPS project for our exchange semester at CY-Tech, also a very thankful for the support, guidance and supervision on our semester progress from the international students office directed by Mr. Michel Guilmault , also Mrs. Audrey Jama and Mrs. Vanessa Meguellati.

June ' 2026

Abstract

Accurate short-term rainfall prediction is of direct practical value to agriculture, transportation, water management, and everyday planning, yet remains challenging due to the noisy, multi-sensor, and class-imbalanced nature of historical weather data. This report presents an end-to-end machine learning pipeline for next-day rain prediction built on the “Rain in Australia” dataset (145,460 daily observations from 49 weather stations, 2007–2017). Following exploratory data analysis, the dataset was cleaned via median/mode imputation and interquartile-range outlier capping, and enriched through twelve engineered features capturing intraday temperature, humidity, and pressure dynamics, expanding the feature space from 23 to 35 columns. Four classifiers; Random Forest, a linear Support Vector Machine, a Gated Recurrent Unit (GRU) neural network, and LightGBM, were trained and evaluated on the resulting dataset, each paired with a class-imbalance strategy (SMOTE oversampling or class-weighted loss) suited to its architecture. Evaluated on a held-out test set, LightGBM achieved the strongest overall performance (85.0% accuracy, 64.9% precision, 68.7% recall, 66.8% F1 score, 0.891 ROC-AUC), outperforming Random Forest (85.2% accuracy, 61.4% F1, 0.885 AUC), the Gated Recurrent Unit (80.1% accuracy, 63.6% F1, 0.881 AUC), and the Support Vector Machine (75.3% accuracy, 58.9% F1, 0.857 AUC), and was selected as the production model. SHAP analysis of the deployed LightGBM model identified afternoon humidity, wind gust speed, and same-day rainfall as the dominant predictors of next-day rain, consistent with established meteorological theory. The trained model was deployed in an interactive Streamlit web application, allowing users to input current weather conditions and receive a calibrated rain probability with corresponding advice. Direct end-to-end testing of this deployment, however, uncovered three feature-encoding defects in the application layer, mismatched location and wind-direction codes and a silently dropped date feature, that compromise the reliability of live predictions despite the underlying model's strong offline performance, underscoring the importance of validating deployed inference pipelines against their training-time preprocessing, not just the trained model in isolation.

Table of Contents

	Page
List of Figures	5
List of Tables	5
List of Abbreviations	5
1 Introduction	6
2 Exploratory Data Analysis - Data Cleaning	8
3 Feature Engineering	12
4 Evaluation Metrics	19
4.1 Accuracy	19
4.2 Precision	20
4.3 Recall	21
4.4 F1 Score	22
4.5 Confusion Matrix	22
4.6 ROC Curve	22
5 Applied Machine Learning & Deep Learning Models	25
5.1 Random Forest Model	25
5.2 Support Vector Machine	26
5.3 Gradient Boosting : LightGBM	26
5.4 Gated Recurrent Unit (GRU)	26
6 Results Interpretation & Analysis	27
6.1 Understanding Results	26
6.2 Metrics Results Comparison & Confusion Matrix	26
6.3 Feature Importance & SHAP Graph	27
7 Conclusion	27

List of Figures

Fig.1 Showing the Pearson Correlation for the features

Fig.2 Showing the distribution for the target feature (RainTomorrow)

Fig.3 Showing the distribution for the target feature (RainTomorrow)

Fig.4 Showing the missing values map

Fig.5 Showing Distribution Histograms of Numerical Features

Fig.6 Showing the correlation matrix between the features

Fig.7 Original vs. Log-Transformed Rainfall Distribution

Fig.8 Showing ROC Curve for Random Forest Model

Fig.9 Showing Precision – Recall Curve for Random Forest Model

Fig.10 Showing Confusion Matrix for the Random Forest

Fig.11 Showing the Confusion Matrix and ROC Curve for the SVM model

Fig.12 Showing the Target class distribution before and after SMOTE

Fig.13 Showing the ROC curve for LightGBM

Fig.14 Showing the Precision – Recall Curve for LightGBM

Fig.15 Showing the confusion matrix for the LightGBM

Fig.16 Showing the Confusion Matrix and ROC curve for the GRU Model

Fig.17 Showing Comparison between all the used models metrics

Fig.18 showing the LightGBM Feature Importance graphs

Fig.19 Showing the SHAP graph for the LightGBM model

List of Tables

Table 1 IQR Outlier Capping Bounds and Number of Values Capped

Table 2 Engineered Features Added During Feature Engineering

Table 3 SVM — Configuration and Final Test Metrics

Table 4 Architecture Search and Final Test Metrics for the GRU model

Table 5 Showing Comparison between all the used models metrics

List of Abbreviations

Abbreviation	Meaning
AUC	Area Under the Curve
CV	Cross-Validation
EDA	Exploratory Data Analysis
FN	False Negative
FP	False Positive
GRU	Gated Recurrent Unit
IQR	Interquartile Range
LightGBM	Light Gradient Boosting Machine
ML	Machine Learning
DL	Deep Learning
PCA	Principal Component Analysis
PR Curve	Precision-Recall Curve
RF	Random Forest
ROC	Receiver Operating Characteristic
SHAP	Shapley Additive Explanations
SMOTE	Synthetic Minority Over-sampling Technique
SVM	Support Vector Machine
TN	True Negative
TP	True Positive
FP	False Positive
FN	False Negative
UI	User Interface

1- Introduction

Rainfall forecasting is one of the most significant important challenges in applied meteorology, as artificial intelligence solutions have been solutions approaches in many challenges through different machine learning and deep learning solutions, it was an effective approach to help in the rainfall predictions for Australia according to dataset that was published on Kaggle. Accurate short term rain prediction supports many decisions on different fields like agriculture , transportation water resources management , and even daily life personal routine for the people. This project has addressed specific tasks of predicting the probability of raining of the next day gives a set of meteorological observations recorded on the given dataset.

The dataset used throughout the project published publicly as “Rain in Australia” dataset (weatherAUS.csv) as its source is Kaggle, it is containing daily weather observations collected from 49 weather stations across all Australia between 2007 and 2017 as total records are 145,460 and 23 raw columns, including temperature, rainfall, evaporation, sunshine, wind, humidity, pressure, and cloud cover measurements at two times of day (9am and 3pm), together with the binary target variable RainTomorrow.

The dataset used throughout this project is the publicly available “Rain in Australia” dataset (weatherAUS.csv), sourced from Kaggle. It contains daily weather observations collected from 49 weather stations across Australia between 2007 and 2017, totaling 145,460 records and 23 raw columns, including temperature, rainfall, evaporation, sunshine, wind, humidity, pressure, and cloud cover measurements at two times of day (9am and 3pm), together with the binary target variable RainTomorrow, the dataset is classified as tabular dataset with a goal binary classification and after the preprocessing stage and feature engineering the dataset was having 25 features as 20 of them were originally in the dataset and 5 features were added by feature engineering process and shown the Pearson Correlation for them in Fig.1, also the dataset is imbalanced dataset as the target “Rain Tomorrow” is having 113,583 as “No Rain” and on the opposite it is having 31,877 as “ Rain” as the Fig. 2 showing.

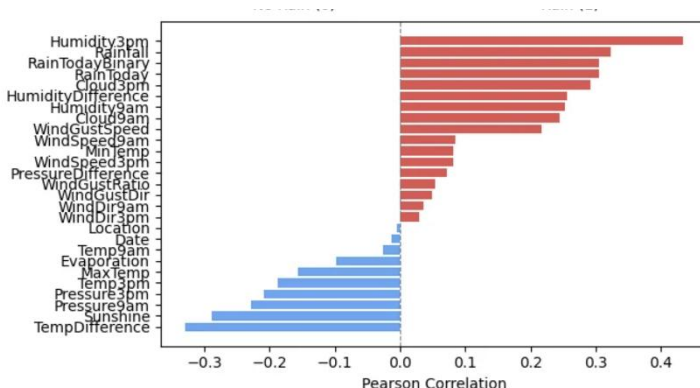


Fig.1 Showing the Pearson Correlation for the features

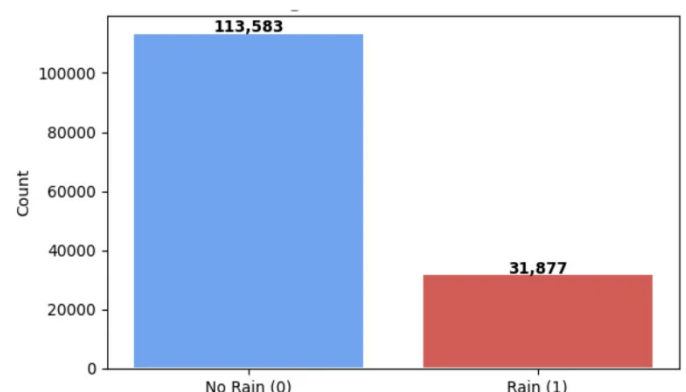


Fig.2 Showing the distribution for the target feature (RainTomorrow)

The Project has followed a complete end to end machine learning pipeline ; starting with the part for exploratory data analysis (EDA), cleaning and preprocessing the dataset , then it was followed by feature engineering part to construct additional information from the data in the form of new features , then we went through training and tuning part, as we started examining four models to see the variety of their results : Random Forest, Support Vector Machine (SVM), and LightGBM and the fourth one was deep learning model which is Gated Recurrent Unit (GRU), then we started the deployment stage depending on the best-performing model into an interactive web application built with Streamlit.

The objectives of our project were:

- To prepare a clean, preprocessed , feature engineered dataset ready to be trained as binary classification supervised data; by addressing the missing values, balancing the data ratios, managing outliers and encoding the categorial variables.
- To train, tune and compare different machine learning and deep learning models for the next day rain predictions.
- To select the best performing model based on different metrics and deploy a full prediction web application through Streamlit with a user friendly interface ready to use depending on the best performance trained model.

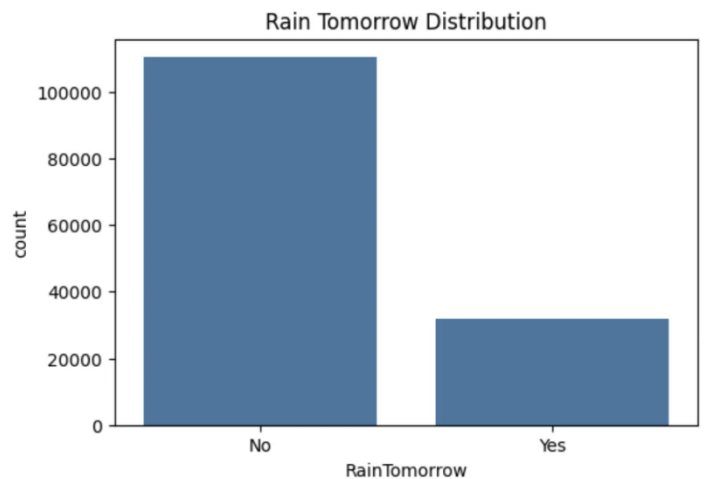
2- Exploratory Data Analysis - Data Cleaning

An essential stage before any training or before any modelling could take place, the raw weatherAUS.csv dataset was explored and cleaned, as this stage had two goals: first, to understand the structure, quality, and statistical characteristics of the data and to get the relations between the features for the data to make better connections between the features and second, to repair the issues that would otherwise compromise model training, namely missing values, duplicate records, and extreme outliers.

• Dataset Overview

Inspection of the target variable revealed a significant class imbalance: rainy days (RainTomorrow = Yes) are considerably less frequent than non-rainy days (RainTomorrow = No), with roughly 22% of records labelled as rain and 78% as no rain (excluding missing target values). This imbalance was a critical consideration carried forward into every subsequent modelling stage, motivating the use of resampling techniques such as SMOTE technique.

Fig.3 Showing the distribution for the target feature (RainTomorrow)



• Missing Values Analysis

Missing values were present across almost every column. The percentage of missing values per column was computed and ranked, and visualized collectively using a missing-values heatmap.

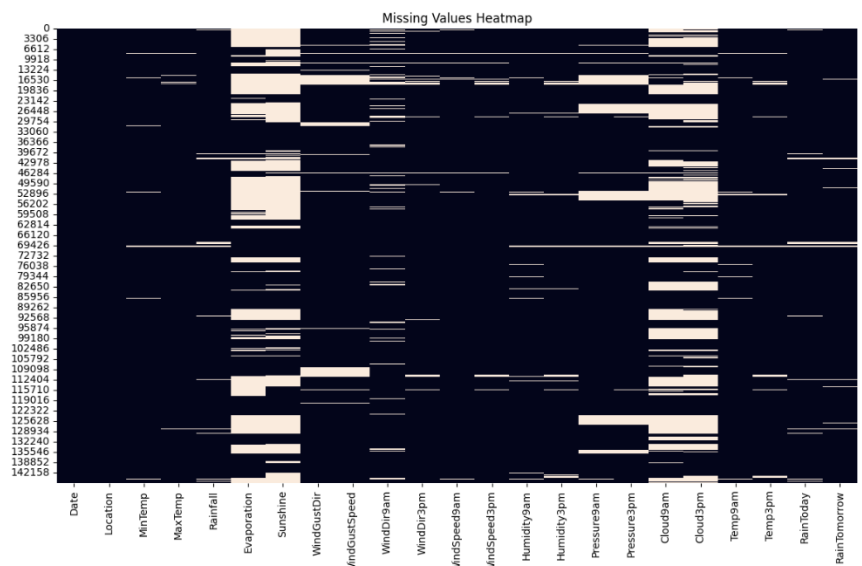


Fig.4 Showing the missing values map

• Distribution and Outlier Analysis

Each numerical feature was visualized using histograms with kernel density estimates (sns.histplot with kde=True) to inspect distribution shape and skewness, and boxplots (sns.boxplot) to visually identify outliers and assess spread. The 16 individual plots produced by the original script have been consolidated into the two grids below for presentation.

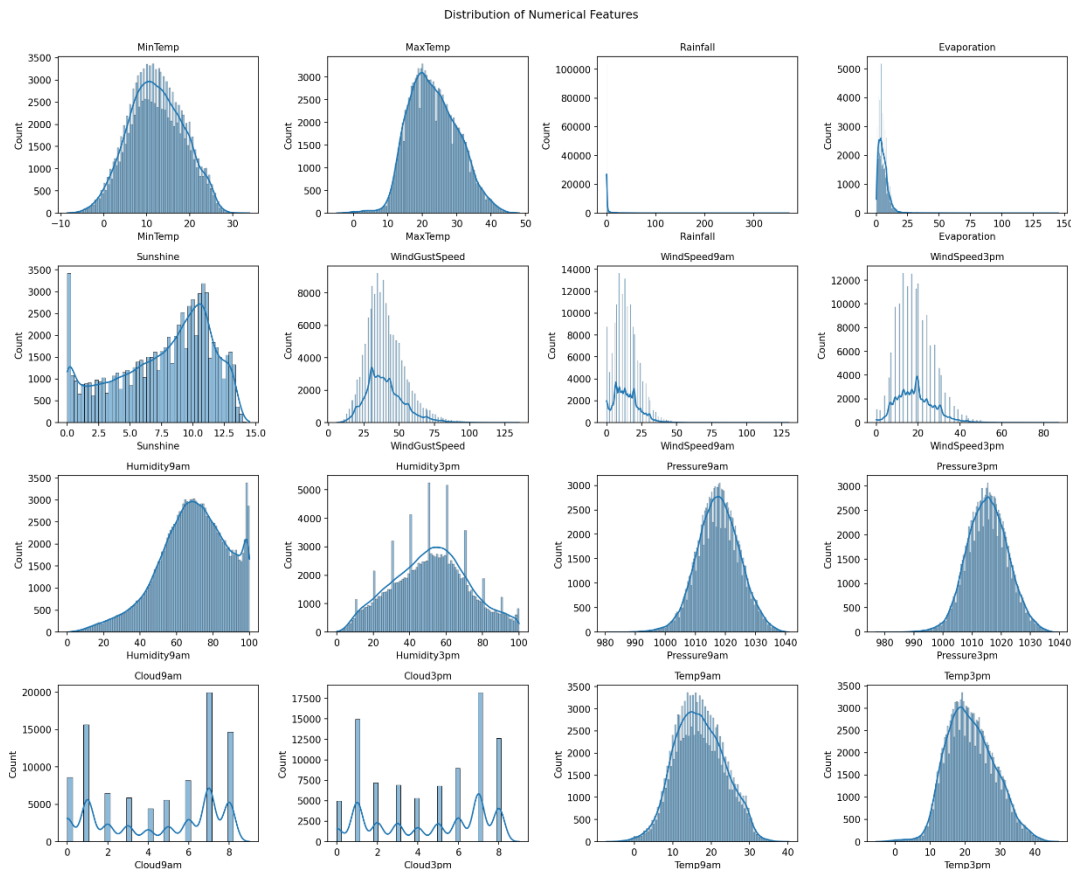


Fig.5 Showing Distribution Histograms of Numerical Features

Several distinct patterns are visible. MinTemp, MaxTemp, Temp9am, and Temp3pm are approximately bell-shaped and only mildly skewed. Rainfall is extremely right-skewed, with the overwhelming majority of days recording zero or near-zero rainfall and a long tail of rare, high-rainfall events. Pressure9am and Pressure3pm are tightly and symmetrically distributed around roughly 1,015–1,016 hPa, consistent with the relatively narrow real-world range of atmospheric pressure at sea level. Cloud9am and Cloud3pm, being recorded in discrete oktas (eighths of sky cover, 0–8), naturally appear as a set of discrete bars rather than a smooth distribution.

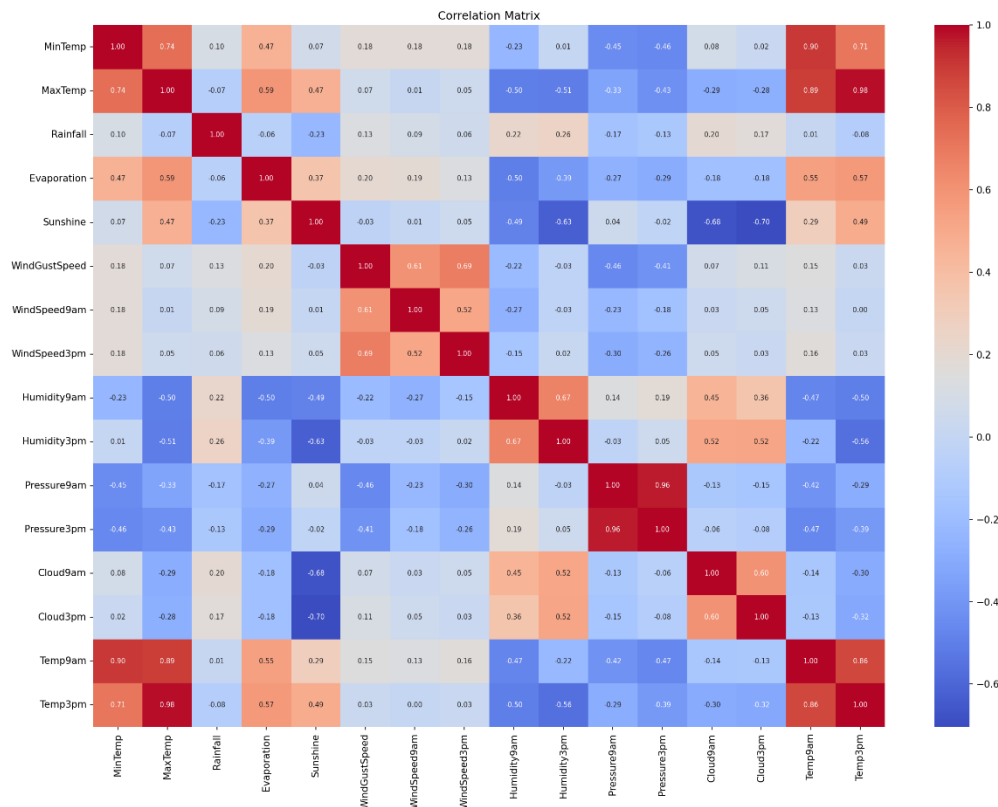


Fig.6 Showing the correlation matrix between the features

The strongest correlations in the dataset are, unsurprisingly, between physically related quantities: Pressure9am and Pressure3pm correlate at 0.96, MaxTemp and Temp3pm at 0.98, MinTemp and Temp9am at 0.90, and MinTemp and MaxTemp at 0.74. Several informative negative correlations also stand out: Sunshine correlates at -0.68 with Cloud9am and -0.70 with Cloud3pm (more cloud cover means less sunshine, as expected), and Sunshine correlates at -0.63 with Humidity3pm. Humidity9am and Humidity3pm correlate positively with each other (0.67) and both correlate negatively with Pressure9am/Pressure3pm and Sunshine, consistent with the humidity/pressure relationship visualised in the scatter plot above. Rainfall itself shows only weak linear correlations with any single numerical feature (the strongest being 0.26 with Humidity3pm), which helps explain why rain prediction benefits from non-linear, multi-feature models such as Random Forest and LightGBM rather than simple linear relationships.

Column	Lower Bound	Upper Bound	Capped Low	Capped High
MinTemp	-5.95	30.45	71	11
MaxTemp	2.70	43.50	360	184
Rainfall	-0.90	1.50	0	28,938
Evaporation	2.20	7.00	15,190	22,285
Sunshine	7.45	9.45	31,690	30,166
WindGustSpeed	8.50	68.50	20	5,503
WindSpeed9am	-11.00	37.00	0	1,817
WindSpeed3pm	-3.50	40.50	0	2,523
Humidity9am	18.00	122.00	1,425	0
Humidity3pm	-5.00	107.00	0	0
Pressure9am	1001.05	1034.25	1,593	1,165
Pressure3pm	998.65	1031.85	1,351	1,173
Cloud9am	-1.50	10.50	0	0
Cloud3pm	1.00	9.00	4,974	0
Temp9am	-1.50	35.30	203	104
Temp3pm	2.45	40.45	452	536

Table 1 IQR Outlier Capping Bounds and Number of Values Capped

Rainfall stands out sharply: 28,938 records (just under 20% of the dataset) exceeded the upper IQR bound of 1.50mm and were capped, confirming the strongly right-skewed distribution observed in this dataset, the vast majority of days have very low rainfall, so even a modest amount of rain registers as a statistical outlier under the IQR rule. Sunshine and Evaporation were also heavily capped on both tails (over 60,000 and 37,000 combined adjustments respectively), reflecting their wide real-world range and the bimodal Sunshine distribution noted earlier. Humidity3pm and Cloud9am, in contrast, had zero values capped on either side, indicating their full observed range already falls within 1.5 IQR of the quartiles.

- **Data Cleaning Procedures**

Following the exploratory phase, the dataset was systematically cleaned as follows:

- Missing value imputation: numerical columns were imputed using the median of each column chosen for its robustness to outliers and skewed distributions. Categorical columns were imputed using the most frequent category. After this step, every column reached zero missing values.
- Duplicate removal: exact duplicate rows were checked, In this run, zero duplicate rows were found both before and after the check, indicating the raw dataset contained no exact duplicate records.
- Outlier capping: for every numerical column, the Interquartile Range (IQR) method was applied. Values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ were capped (clipped) to the corresponding bound, rather than removed, to preserve sample size while limiting the influence of extreme values on model training.

3- Feature Engineering

Following data cleaning, a dedicated feature engineering stage was carried out within the same script (`data_preprocessing_feature_engineering.py`) to enrich the dataset with derived variables, encode categorical information numerically, and place all features on a comparable scale. This stage transformed the cleaned 23-column raw dataset into a 35-column engineered dataset (145,460 rows $\times$ 35 columns) suitable for direct consumption by all four downstream models.

- **Target-Adjacent Transformation**

Rainfall (mm) was log-transformed using a log1p transformation ($\text{Rainfall_Log} = \log(1 + \text{Rainfall})$), intended to compress its long right-skewed tail and produce a more symmetric, model-friendly distribution.

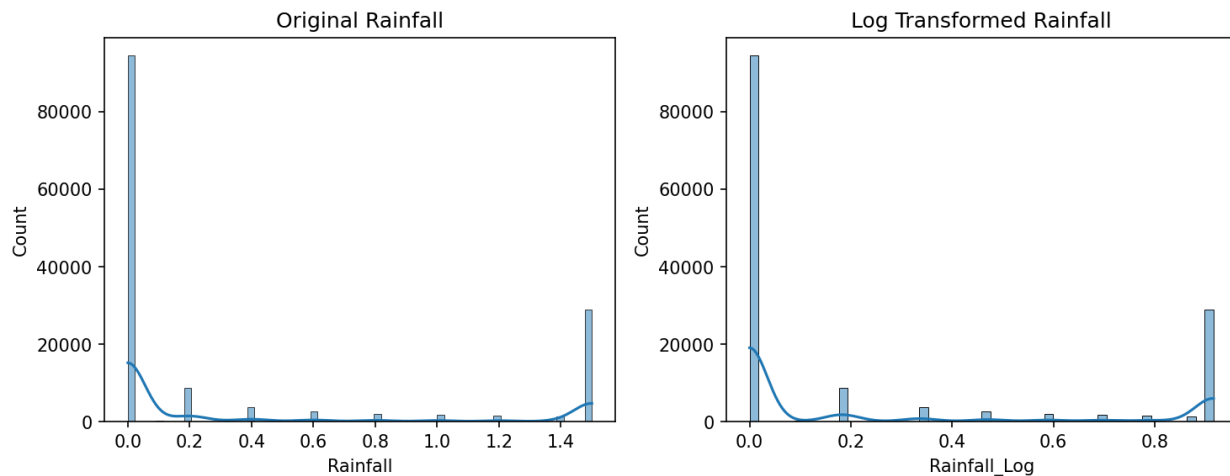


Fig.7 Original vs. Log-Transformed Rainfall Distribution

In practice, because the outlier-capping step had already clipped Rainfall to a maximum of 1.50mm (capping 28,938 of the original values), the variable's range was already heavily compressed before the log transform was applied. As a direct consequence, the two histograms above are visually almost identical: the log transform has very little additional effect once the extreme values have already been capped. This is a genuine finding from running the pipeline as written, and it highlights an ordering consideration for this kind of pipeline, log-transforming Rainfall before IQR-capping it (rather than after) would likely have preserved more of the transformation's intended benefit, since the capping bounds would then have been computed on the already-compressed log scale rather than on the raw, heavily-skewed scale.

- **Categorical Encoding**

The binary categorical column RainToday was additionally mapped directly to a numeric binary indicator (RainTodayBinary), with 'Yes' mapped to 1 and 'No' mapped to 0. Separately, all object-dtype (categorical) columns at that point in the script: Date, Location, WindGustDir, WindDir9am, WindDir3pm, RainToday, and RainTomorrow, were transformed, converting each category into an integer code.

- **Derived Meteorological Features**

A set of domain-informed derived features was constructed to capture relationships between paired 9am/3pm measurements and between related physical quantities, on the premise that the rate of change and interaction between variables over the course of a day carries additional predictive signal beyond the raw point measurements themselves.

Feature	Formula	Rationale
TempDifference	$\text{MaxTemp} - \text{MinTemp}$	Daily temperature range/volatility
HumidityDifference	$\text{Humidity3pm} - \text{Humidity9am}$	Intraday humidity trend
PressureDifference	$\text{Pressure3pm} - \text{Pressure9am}$	Intraday pressure trend (a key rain precursor)
WindSpeedInteraction	$\text{WindSpeed9am} \times \text{WindSpeed3pm}$	Combined wind activity across the day
WindGustRatio	$\text{WindGustSpeed} / (\text{WindSpeed3pm} + 1)$	Gustiness relative to sustained wind
AvgTemp	$(\text{MinTemp} + \text{MaxTemp}) / 2$	Representative daily temperature
AvgHumidity	$(\text{Humidity9am} + \text{Humidity3pm}) / 2$	Representative daily humidity
AvgPressure	$(\text{Pressure9am} + \text{Pressure3pm}) / 2$	Representative daily pressure
TempHumidityIndex	$\text{AvgTemp} \times \text{AvgHumidity}$	Joint heat-moisture interaction
PressureHumidityInteraction	$\text{AvgPressure} \times \text{AvgHumidity}$	Joint pressure-moisture interaction

Table 2 Engineered Features Added During Feature Engineering

4- Evaluation Metrics

Before interpreting any model output, we had to define what success actually means for this problem. In a strongly imbalanced setting, the choice of evaluation metric is not a formality. The choice is based on a design decision that determines whether the resulting model has any real-world value at all.

Accuracy: The “Trap” Metric

Accuracy measures the overall percentage of correct forecasts. While intuitive, it is dangerously misleading on our dataset. Because the class distribution is not balanced and it is structured with 78% "No Rain" vs. 22% "Rain", a standard model that predicts "No Rain" every single day would achieve close to 78% accuracy while being completely useless. This embraced a fundamental problem because it would never warn anyone of an incoming storm. For this reason, accuracy was not treated as the most important metric.

Precision: The Cost of False Alarms

Precision answers the question: when the model predicts "Rain," how often does it actually rain? In business terms, low precision means a high rate of False Alarms. In our case, with a low level of precision, the model would warn "storm," people would cancel outdoor plans, reschedule flights or postpone agricultural operations, but in reality the sun shines all day. Beyond the direct economic loss, repeated false alarms erode user trust, leading people to ignore the forecast entirely.

Recall: The Cost of Missed Storms

Recall answers the complementary question: out of all the days it actually rained, how many did we successfully catch? Low recall means Missed Storms (False Negatives). The model predicts clear skies, people leave their umbrellas at home, skip precautions or proceed with field work and then get caught in a downpour. In high-stakes domains such as flood warning or aviation safety, a missed storm is far more costly than a false alarm, where it can occur on expensive countermeasures implementation.

F1-Score: Our Guiding Metric

Precision and Recall are interlinked and correlate: pushing one up drags the other down. The F1-Score, defined as the harmonic mean of the two, captures this trade-off in a single number. The harmonic mean is chosen over a simple average because it penalizes extreme imbalances (lower score among the two), a model cannot inflate its F1 by sacrificing one metric for the other; a solid F1 rank is justified only when both Precision and Recall are reasonably high together.

ROC Curve & AUC: Ranking Quality Across All Thresholds

The metrics above (Precision, Recall, F1) all share one hidden limitation: they are measured at a single fixed decision threshold. To understand why this matters, we need to separate two distinct steps. First, the model looks at the weather conditions and outputs a probability, a number between 0 and 1. To convert that probability into a label, we apply a threshold: above it we predict "Rain," below it "No Rain". We choose where to place that cut, and the same probability can become "Rain" or "No Rain" depending on it. This is exactly the limitation the ROC Curve (Receiver Operating Characteristic) overcomes: instead of judging the model at one arbitrary cut, it shows its behavior at every possible threshold. It plots the True Positive Rate against the False Positive Rate as the cutoff moves from 0 to 1. The diagonal line is random guessing. The more the curve bends toward the top-left corner, the better the model separates rainy days from dry days. The AUC (Area Under the Curve) turns this whole curve into a single number: it is the probability that the model gives a higher score to a random rainy day than to a random dry day.

Precision-Recall Curve: The Right Lens for Imbalanced Data

The ROC Curve is useful, but on imbalanced data it can look too optimistic. The reason is that the False Positive Rate is "diluted" by the huge number of "No Rain" days, which the model classifies correctly very easily. The Precision-Recall Curve fixes this blind spot: it ignores true negatives completely and plots Precision against Recall across thresholds. This keeps the focus only on the rare and important "Rain" class. Its baseline is not a diagonal but a flat horizontal line equal to the share of positive cases in the data. Any curve clearly above that line shows real predictive skill, beyond simply guessing the base rate.

5- Applied Machine Learning & Deep Learning Models

Four predictive models were trained and evaluated on the cleaned, engineered dataset; Random Forest, linear Support Vector Machine (SVM), Gated Recurrent Unit (GRU) neural network, and LightGBM.

SMOTE Analysis The Problem: A Model Biased Toward "No Rain"

Our training data was strongly imbalanced: there were about 3.56 "No Rain" days for every "Rain" day. As mentioned in the previous chapter, this is a real problem, because a model can reach a high accuracy just by almost always predicting "No Rain". To give the model a fair chance of learning the patterns that come before rain, we needed to balance the two classes during training. Our Solution: Synthetic Oversampling We used SMOTE (Synthetic Minority Over-sampling Technique). SMOTE does not simply copy the rain days, because copying would push the model to memorize them. Instead, it creates new synthetic rain examples by interpolating between real minority samples and their nearest neighbours in the feature space. This brought the training set to a 1:1 ratio, so the "Rain" class had the same number of samples as the "No Rain" class. The test set was left completely untouched, so it still reflects the real frequency of rain. Application of the model: No Data Leakage The key choice was where we applied SMOTE. If we balance the whole dataset before the split or before cross-validation, information from the validation data leaks into the training, and the scores become inappropriate. So we never applied SMOTE to the full dataset. We applied it only to the training data, and kept the test data untouched, so it still shows the real frequency of rain. When cross-validation was used, SMOTE was fitted only on the training part of each fold, never on the part used for testing. This way the balancing happens inside the training, not before it, and our results stay robust.

Random Forest Model

We started with a Random Forest as our baseline model. A Random Forest is an ensemble of many decision trees: each tree is trained on a random part of the data and gives its own vote, and the final prediction is the majority vote of all the trees. We chose it for two main reasons. First, it is robust: by averaging many trees it is naturally resistant to outliers and to overfitting, which makes it a solid reference point to compare the other models against. Second, it captures non-linear patterns, so it can pick up combinations like specific humidity and pressure values that together trigger rain, something that linear model would never achieve.

Hyperparameter Tuning

To find the best settings without guessing, we used RandomizedSearchCV. Testing every possible combination (a full Grid Search) would be very expensive in terms of computational time: each combination requires training the whole forest from scratch, and with cross-validation this training is repeated once per fold, so the CPU cost grows quickly. RandomizedSearchCV avoids this by trying only a fixed number of random combinations and

keeping the best one, still using cross-validation to make sure the choice is stable and not just lucky on one split. We tuned the number of trees, the maximum depth, and the minimum samples needed to split a node, and we optimized for the F1-Score so the search would care about the rain class and not just overall accuracy. The best configuration used 300 trees with a minimum split of 10, reaching a cross-validation F1 of about 0.64 on the rain class.

Overfitting Analysis

After tuning, we checked for overfitting by comparing the accuracy on the training set with the accuracy on the test set. The model scored 98.56% on training but only 84.51% on the test set, a gap of about 14%. This gap tells us the model partly memorized the historical data instead of fully learning the general weather patterns. This is a known behaviour of deep trees: when they are allowed to grow large, they fit the training data almost perfectly. Even with this gap, the accuracy on the test set stayed high and stable, showing that the model still generalizes well to unseen data and gives reliable predictions.

Probability Calibration (Platt Scaling)

This last step was the most important one to make the model usable in the real world, not just statistically accurate. What is the problem? A Random Forest does not compute a probability directly. It builds many trees (200 in our case) and calculates the probability of rain as the average of their votes. Since every tree is trained on a different random part of the data, the trees almost never agree 100%. Because of this averaging, the probabilities get "squashed" toward the middle: the model rarely outputs values close to an absolute 0% or 100%.

The calibrated Random Forest reached 85 percent accuracy on the test set. The more important result was the area under the ROC curve of 0.886, which means that given a random rain day and a random dry day, the model ranks the rain day higher about 89 percent of the time. This was a key metric because it measures how well the model separates rain days from dry days, independently of the threshold. On the ROC curve, the orange line bends sharply toward the top-left corner, far above the dashed diagonal of a random classifier.

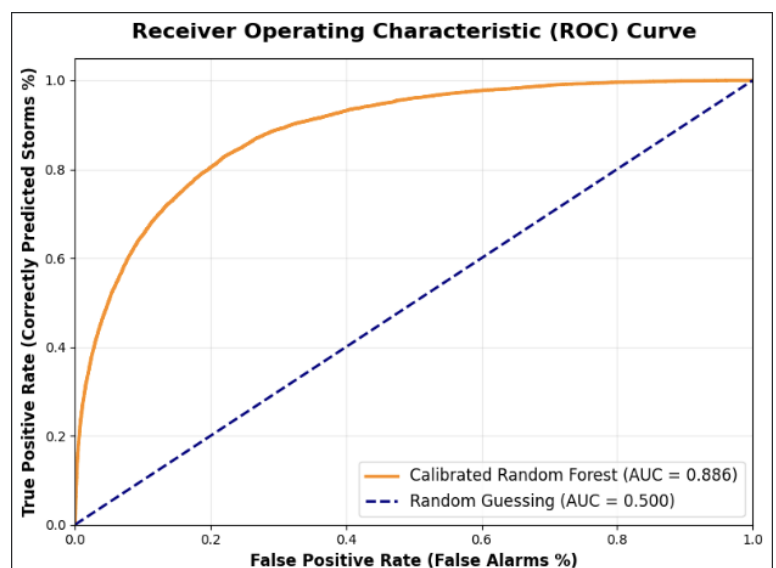


Fig.8 Showing ROC Curve for Random Forest Model

On the rain class, which is the class that matters most, the model reached a precision of 0.72 and a recall of 0.53, for an F1 score of 0.61. The precision of 0.72 was the strong point: when the model predicts rain, it is right about 72 percent of the time, so it produces few false alarms. The recall of 0.53 means it catches a bit more than half of the real rain days, so the model is precise but cautious. The precision-recall curve confirms this: it stays well above the baseline of 0.221, which equals the real share of rain days in the data, and holds a high precision when the model is most confident, which shows that a high probability output can be trusted.

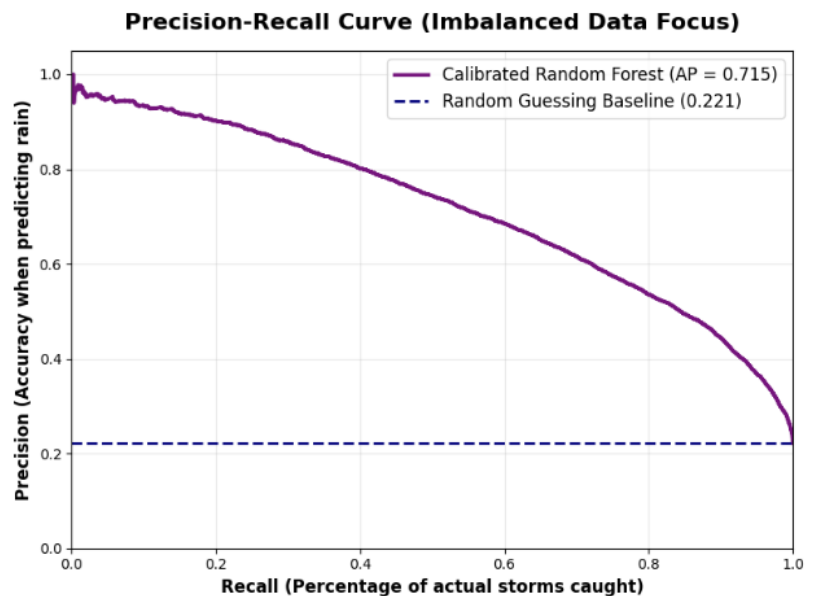


Fig.9 Showing Precision – Recall Curve for Random Forest Model

The confusion matrix shows the same picture in absolute numbers. The model correctly identified 32,030 dry days and 5,143 rain days, against 1,957 false positives, where it predicted rain but the day stayed dry, and 4,508 false negatives, where it predicted no rain but rain actually came. The very low number of false positives, compared to the large number of correctly classified dry days, matches the high precision of the model, while the higher count of false negatives reflects the lower recall, confirming that the Random Forest is precise but cautious in predicting rain.

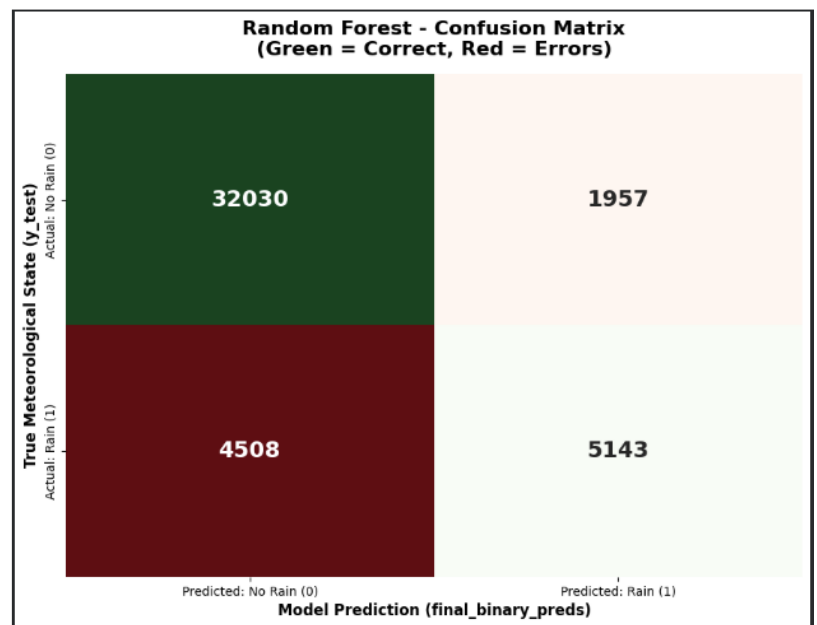


Fig.10 Showing Confusion Matrix for the Random Forest

Support Vector Machine

The SVM pipeline used an 80%/20% stratified split, SMOTE oversampling of the training set, and PCA retaining 95% of variance — compressing the 34 standardized features to 16 principal components (95.44% variance retained). A LinearSVC (C=1.0, class_weight='balanced') wrapped in CalibratedClassifierCV was validated with 5-fold stratified cross-validation, then evaluated at a lowered 0.45 decision threshold to favour Recall.

Table 3 SVM — Configuration and Final Test Metrics

Metric	Value
PCA components / variance retained	16 of 34 features / 95.44%
5-fold CV mean accuracy	0.7745
Train / Test accuracy	77.45% / 77.75% (gap = 0.3 pp — well generalized)
Final Accuracy / Precision / Recall / F1	75.31% / 46.36% / 80.75% / 58.90%
ROC-AUC / Average Precision	0.857 / 0.668

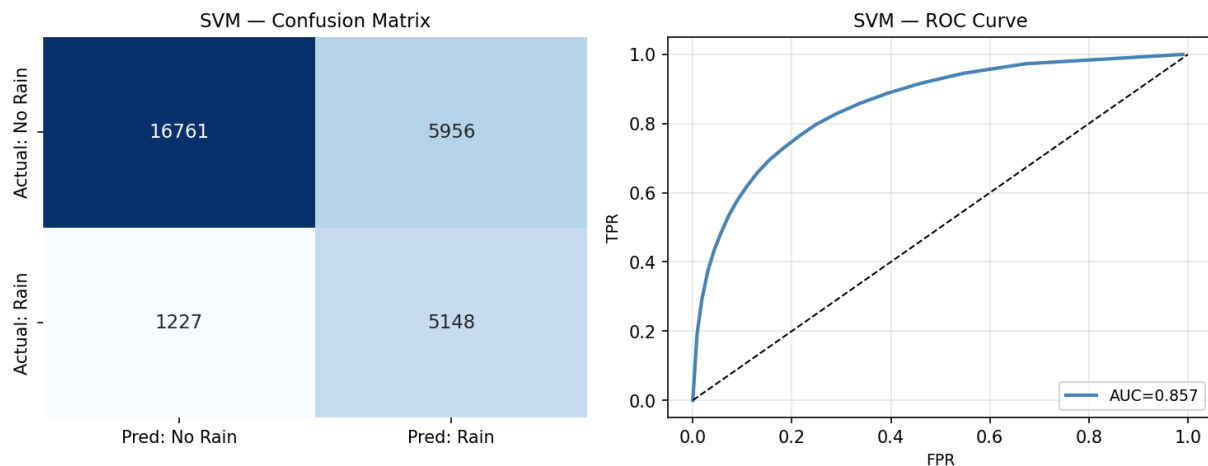


Fig.11 Showing the Confusion Matrix and ROC Curve for the SVM model

Unlike Random Forest, SVM shows almost no train/test gap, reflecting the regularizing effect of its linear decision boundary and the dimensionality reduction from PCA. This stability comes at a real cost: SVM achieves the highest Recall of all four models (80.75%, catching 5,148 of 6,375 rain events) but the lowest Precision (46.36%) and lowest overall Accuracy (75.31%), raising 5,956 false alarms in the process — a direct, measurable consequence of combining class-balanced weighting with SMOTE on a linear model that cannot draw a non-linear decision boundary around the minority class.

Gradient Boosting : LightGBM

One of the central modeling approaches in this project was Gradient Boosting. Gradient Boosting builds many small decision trees sequentially where each new tree corrects the mistakes of the previous one and after 200 trees, they combine into one powerful predictor. Among the three gradient boosting variants we tested, LightGBM gave the strongest and most stable results, becoming the recommended model.

Gradient Boosting suits our project problem because of the nature of weather itself. Rain doesn't depend on any single variable but it depends on combinations. High afternoon humidity means little on its own but combined with a falling pressure trend or strong wind gusts it reliably signals that rain is coming. Gradient boosting learns through decision trees that naturally capture thresholds and interactions, which is exactly the structure weather data contains. LightGBM was the best over standard gradient boosting and XGBoost because it was the fastest of the three and the best suited to a large table of data. It grew its trees leaf by leaf, expanding the branch that reduces the most error first, which on more than 145,000 rows saved real time and memory while keeping accuracy high.

Model application

The model received the same fair starting point where the final feature set held 25 variables, the 20 original measurements plus the 5 engineered features. The data was split 80 percent for training and 20 percent for testing using a stratified split, which keeps the 78 to 22 class ratio in both halves and produced 116,368 training samples and 29,092 test samples. Stratification mattered because a random split could place too few rain days in the test set and make the evaluation unreliable. All features were standardized so that a large scale variable like pressure, would not dominate the model simply because its numbers are bigger than those of humidity.

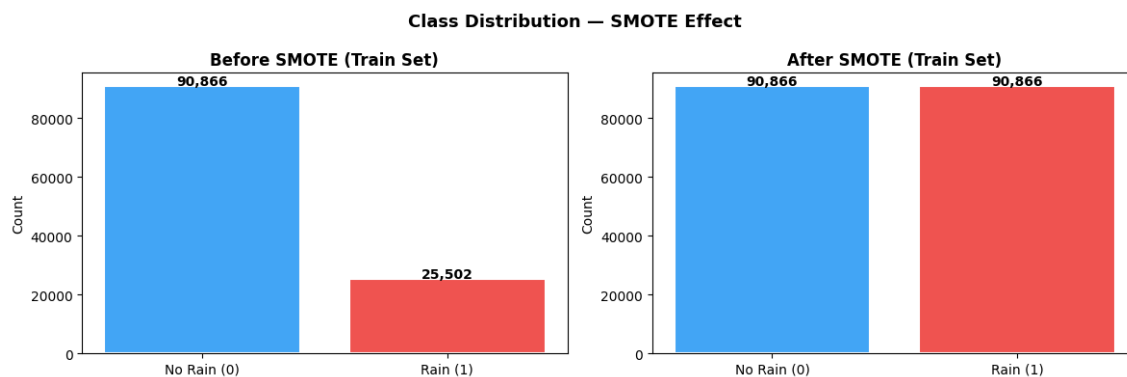


Fig.12 Showing the Target class distribution before and after SMOTE

The class imbalance was handled with SMOTE, applied only to the training set. SMOTE gave the model balanced exposure to the minority class without copying the same rows repeatedly. After SMOTE the training set held an equal number of samples in each class, while the test set was left untouched at its real 78 to 22 distribution so that the evaluation reflected real conditions. Two versions were run, with and without SMOTE, so the effect of balancing could be measured.

The model itself built 200 trees in sequence, each limited to a depth of five so that no single tree grew complex enough to memorize noise. The learning rate was set to 0.1, so each tree added only a small correction and the model learned gradually, and each tree trained on a random 80 percent of the data to add variety and reduce overfitting. LightGBM also applied L1 and L2 regularization.

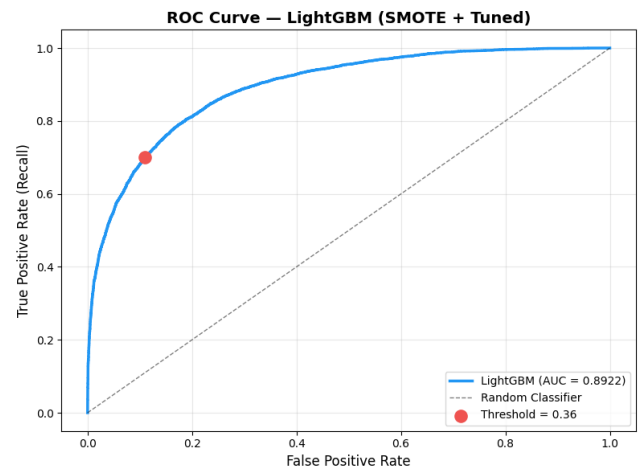


Fig.13 Showing the ROC curve for LightGBM

LightGBM with SMOTE was the best performing model overall. It reached 85% accuracy, but the more important result was the area under the ROC curve of 0,8922 which means that given a random rain day and a random dry day, the model ranks the rain day higher about 89% of the time. This was the main metric because it measured how well the model could separate rain days from dry days.

On the rain class which is the class that matters most, the model reached a precision of 0.64 and a recall of 0.70, for an F1 score of 0.67. The recall of 0.70 means it caught 70 percent of the days that actually turned rainy. The red point on the ROC curve marks this operating point, where the model reaches that 70 percent recall at a false positive rate of only about 11 percent. Lowering the threshold raises recall so fewer real rain events are missed, at the cost of a few more false alarms, which is the right trade for a safety system where missing a storm is worse than a false warning.

The precision recall curve supports this, staying well above the baseline of 0.22 and holding precision close to 1.0 when the model is most confident which shows that a high probability output can be trusted.

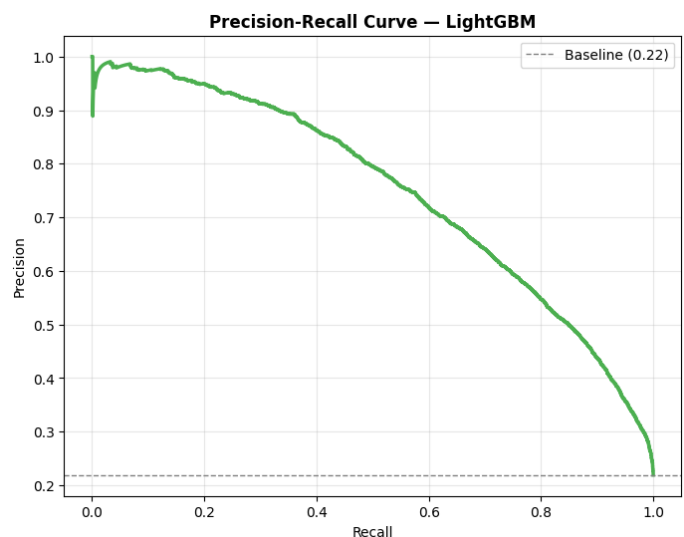


Fig.14 Showing the Precision – Recall Curve for LightGBM

The confusion matrix correctly identified 20,617 dry days and 4,274 rain days, against 2,100 false positives, where it predicted rain but the day stayed dry, and 2,101 false negatives. The fact that these two error counts are almost identical shows that the tuned threshold produced a balanced model that does not systematically favour one type of mistake over the other.

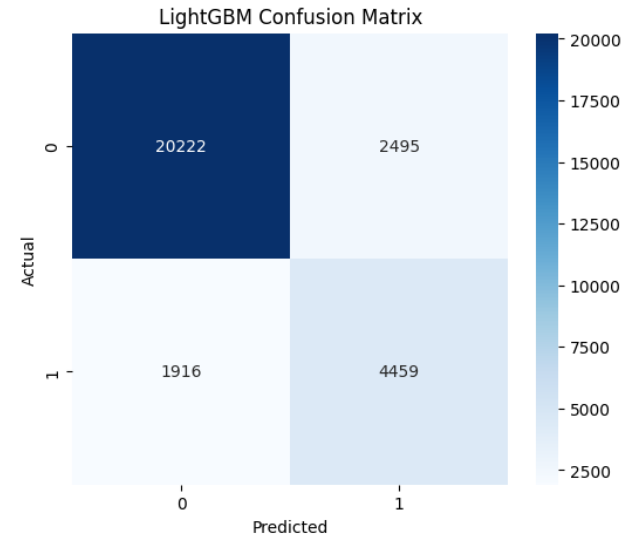


Fig.15 Showing the confusion matrix for the LightGBM

Gated Recurrent Unit (GRU)

Each row of tabular features was reshaped into a length-1 sequence so the GRU architecture (one recurrent layer → Dropout → Dense(32, ReLU) → Dense(1, sigmoid)) could process it; this does not exploit the GRU's native capacity for multi-day temporal sequences. Class imbalance was handled via Keras `class_weight` (balanced: weight 0.640 for No-Rain, 2.282 for Rain) rather than SMOTE. A 2-configuration search (units ∈ {32,64}, dropout=0.3, batch_size=64, 6 epochs, early stopping) selected the best validation-accuracy configuration, which was then retrained with a longer patience for the final model.

Metric	Value
Search results	units=32 → val_acc=0.7950; units=64 → val_acc=0.7973 (selected)
Final configuration	64 GRU units, dropout=0.3, Adam optimiser, binary cross-entropy
Final test loss	0.4232
Final Accuracy / Precision / Recall / F1	80.11% / 53.09% / 79.37% / 63.62%
ROC-AUC / Average Precision	0.881 / 0.720

Table 4 Architecture Search and Final Test Metrics for the GRU model

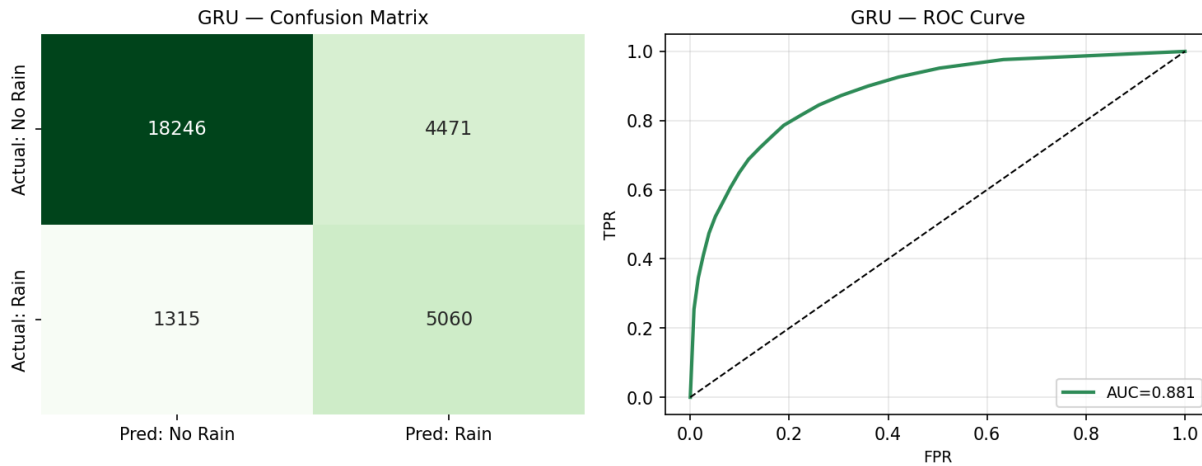


Fig.16 Showing the Confusion Matrix and ROC curve for the GRU Model

GRU's class-weighting strategy (rather than SMOTE) produces a Recall (79.37%) close to SVM's, but with notably better Precision (53.09% vs. 46.36%) and a higher F1 Score (63.62%) and ROC-AUC (0.881) than SVM achieves — the second-best F1 Score among all four models, despite the single-timestep architecture not using any genuine sequential structure. This suggests the non-linear Dense layers, not the recurrent mechanism itself, are doing the discriminative work here; a true multi-day sequence input is listed.

6- Results Interpretation & Analysis

This part synthesizes the four models' results into a single comparison, examines what drives LightGBM's predictions, and reports on direct testing of the deployed application.

Understanding Results

Three considerations frame the comparison that follows. First, class imbalance: Rain is 21.9% of labelled records confirmed at 22.12% and 21.92% in the two held-out test splits used below, so Recall and F1 on the Rain class matter more than raw Accuracy a model predicting 'No Rain' unconditionally would score ~78% accuracy while catching zero rain events. Second, calibration: Random Forest and SVM were explicitly recalibrated (CalibratedClassifierCV) so their predicted probabilities can be trusted downstream, not just their binary decisions. Third, deployability: on this single-core environment, Random Forest's search took ~14.3 minutes, LightGBM's ~8.2 minutes, and the combined SVM+GRU script ~2.1 minutes.

Metrics Results Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest	85.17%	72.31%	53.38%	61.42%	0.885
SVM	75.31%	46.36%	80.75%	58.90%	0.857
GRU	80.11%	53.09%	79.37%	63.62%	0.881
LightGBM (deployed)	85.02%	64.95%	68.74%	66.79%	0.891

Table 5 Showing Comparison between all the used models metrics

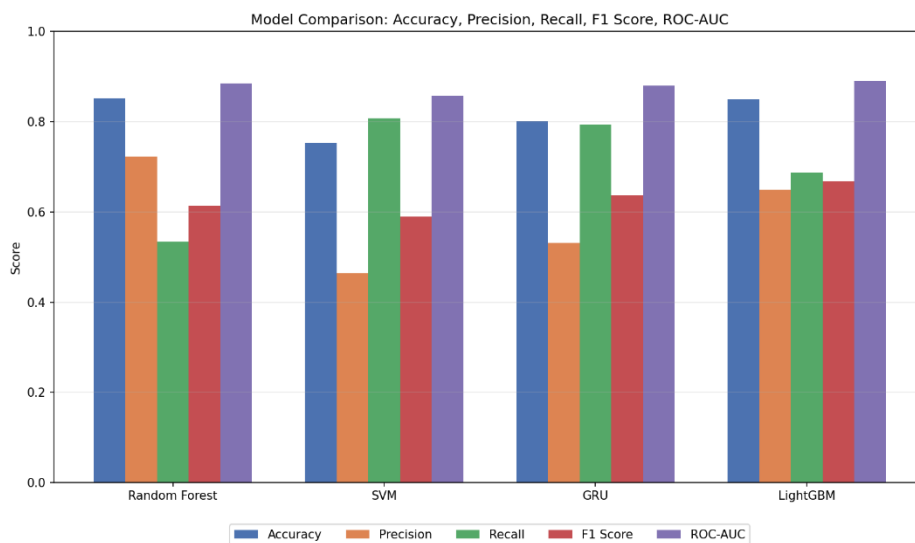


Fig.17 Showing Comparison between all the used models metrics

Feature Importance & SHAP Graph

The two rankings tell different stories. Split-count importance ranks Location first (2,652 splits) because high-cardinality categorical features (49 locations, 16 wind directions) mechanically offer more candidate split thresholds, not because they most influence any single prediction. SHAP — which measures actual contribution to model output — instead ranks Humidity3pm first by a wide margin (mean |SHAP| = 0.848, nearly double the runner-up), with Location falling to 11th (0.162). This matches the raw-data relationship visualized: high afternoon humidity is the dominant physical driver the model has learned, ahead of WindGustSpeed (0.456) and same-day Rainfall (0.382, reflecting real-world multi-day storm persistence).

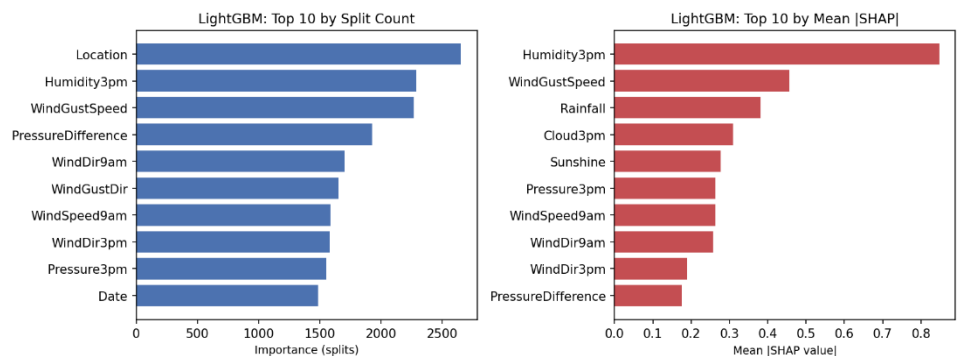


Fig.18 showing the LightGBM Feature Importance graphs

For comparison, SVM's importance and SHAP analyses are expressed over its 16 PCA components rather than original variables, since each component is a linear combination of multiple standardized features; this confirms which combinations matter to the linear boundary but cannot be mapped back to a single physical variable as directly as LightGBM's SHAP results. Because LightGBM is the deployed model, its SHAP ranking — humidity, wind, and rainfall persistence as the strongest drivers — is treated as the primary, decision-relevant finding of this analysis.

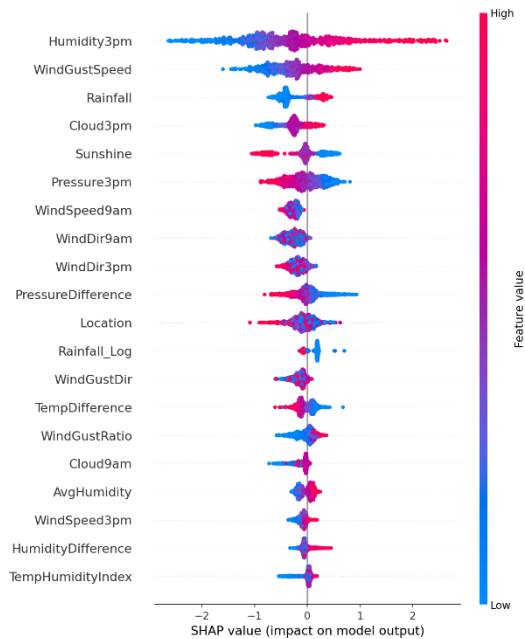


Fig.19 Showing the SHAP graph for the LightGBM model

7- Conclusion

This project built a complete machine learning pipeline to predict whether it will rain the next day in Australia, and turned a noisy, imbalanced weather dataset into a working forecasting system.

A robust end-to-end pipeline

We successfully processed more than 145,000 daily records, handling every problem the raw data presented. We dealt with a large amount of missing values and with extreme outliers through imputation and capping instead of dropping rows, so that we did not lose the rare rain events the model needed to learn from. Above all, we tackled the strong class imbalance of 78 to 22 with SMOTE, applied carefully and only on the training data to keep the evaluation honest. The result is a clean and reliable pipeline that takes raw weather measurements and turns them into trustworthy predictions.

A big part of the model's success came from the engineered features. Instead of feeding the model only the raw measurements, we created new variables that describe the physical behaviour of the weather, such as the change in pressure and humidity between morning and afternoon, and the ratio of wind gusts to baseline wind speed. These features proved their value in practice, as some of them ended up among the most influential variables in the model. This confirmed that capturing how the weather changes during the day matters more than its absolute values, and that these custom indicators helped the models detect the non-linear conditions that trigger rain.

We did not rely on a single algorithm. We trained and compared four different models on the same data: a calibrated Random Forest, a LightGBM gradient boosting model, a Support Vector Machine and a Gated Recurrent Unit network. Each one brought a different strength. The Random Forest offered the highest precision and well-calibrated probabilities, the SVM reached the highest recall on rain events, and the GRU showed the potential of a deep sequential model. Comparing them on the same metrics let us understand the trade-off between catching real rain and avoiding false alarms, and LightGBM stood out as the best overall, with the highest ROC-AUC and the best F1 score on the rain class.

Finally, the project was not only a statistical exercise. We deployed an interactive dashboard where a user can input live weather metrics and test "what-if" scenarios in real time, getting an immediate rain prediction from the model. This turns the pipeline into a practical tool that can support real decisions in fields such as agriculture, transport and public safety, where knowing the chance of rain in advance has a real and concrete value